

Preguntas sobre el ejemplo de clasificación de imágenes con PyTorch y MLP

Integrantes:

- Agostina Almendros (63096)
- Melanie Natasha Mauas (63517)

1. Dataset y Preprocesamiento	1
2. Arquitectura del Modelo	2
3. Entrenamiento y Optimización	2
4. Validación y Evaluación	3
5. TensorBoard y Logging	3
6. Generalización y Transferencia	4
7. Regularización	4
Preguntas teóricas:	4
Actividades de modificación:	5
Preguntas prácticas:	6
8. Inicialización de Parámetros	7
Preguntas teóricas:	7
Actividades de modificación:	8
Preguntas prácticas:	9
Gráficos Finales:	10
Histogramas:	11
Distribuciones:	12

1. Dataset y Preprocesamiento

- **¿Por qué es necesario redimensionar las imágenes a un tamaño fijo para una MLP?**

Porque las MLPs esperan vectores de entrada de tamaño fijo. Como las imágenes tienen dimensiones variables (alto × ancho × canales), necesitamos convertirlas a un tamaño constante para poder aplanarlas (Flatten) y pasarlas por capas lineales.

- **¿Qué ventajas ofrece Albumentations frente a otras librerías de transformación como torchvision.transforms?**
 - Más rápida (usa OpenCV por debajo).
 - Más transformaciones avanzadas (e.g., GridDistortion, RandomFog).
 - Permite composición compleja y probabilística de transformaciones.
 - Soporta numpy, lo que la hace flexible para datasets personalizados.

- **¿Qué hace A.Normalize()? ¿Por qué es importante antes de entrenar una red?**

Estandariza cada canal de la imagen (resta la media y divide por la desviación estándar).

Esto estabiliza el entrenamiento, mejora la convergencia y evita que los gradientes exploten o desaparezcan.

- **¿Por qué convertimos las imágenes a ToTensorV2() al final de la pipeline?**

Porque Albumentations trabaja con numpy arrays, y PyTorch necesita tensores (torch.Tensor). ToTensorV2() convierte imágenes y etiquetas al formato adecuado.

2. Arquitectura del Modelo

- **¿Por qué usamos una red MLP en lugar de una CNN aquí? ¿Qué limitaciones tiene?**

Una MLP es más simple y puede servir para datasets pequeños o problemas de prueba.

Limitaciones: No capta la estructura espacial/local de las imágenes (convoluciones), necesita muchas más conexiones, y no escala bien a imágenes grandes.

- **¿Qué hace la capa Flatten() al principio de la red?**

Convierte la imagen 3D (C, H, W) en un vector 1D para pasar a las capas Linear.

- **¿Qué función de activación se usó? ¿Por qué no usamos Sigmoid o Tanh?**

Se usó ReLU. Es más eficiente computacionalmente y no sufre tanto el problema del desvanecimiento del gradiente, a diferencia de Sigmoid o Tanh. Ambas saturan para valores extremos, lo que lleva a gradientes muy pequeños y aprendizaje lento o nulo en capas profundas.

- **¿Qué parámetro del modelo deberíamos cambiar si aumentamos el tamaño de entrada de la imagen?**

El número de unidades de entrada en la primera capa Linear (porque cambia el tamaño del vector tras el Flatten).

3. Entrenamiento y Optimización

- **¿Qué hace optimizer.zero_grad()?**

Limpia los gradientes acumulados del paso anterior. Si no lo haces, se acumulan.

- **¿Por qué usamos CrossEntropyLoss() en este caso?**

Porque es la función adecuada para clasificación multiclase con etiquetas enteras (class índices), combinando LogSoftmax y NLLLoss.

- **¿Cómo afecta la elección del tamaño de batch (batch_size) al entrenamiento?**
Tamaños grandes generan entrenamientos más estables, pero menos capacidad de generalizar. Uso más eficiente de la GPU, pero mayor consumo de memoria.
Tamaños pequeños generan más ruido, pero mejor exploración del espacio de parámetros.
- **¿Qué pasaría si no usamos model.eval() durante la validación?**
Módulos como Dropout y BatchNorm se comportan diferente en train() que en eval(). No cambiar a eval() lleva a resultados inconsistentes.

4. Validación y Evaluación

- **¿Qué significa una accuracy del 70% en validación pero 90% en entrenamiento?**
Es señal de overfitting: el modelo memoriza el entrenamiento pero no generaliza.
- **¿Qué otras métricas podrían ser más relevantes que accuracy en un problema real?**
Precision, Recall, F1-score
ROC-AUC, PR AUC, dependiendo del problema
- **¿Qué información útil nos da una matriz de confusión que no nos da la accuracy?**
Muestra cómo se confunden las clases. Es útil para detectar errores sistemáticos. De esta matriz además se obtienen las otras métricas como precisión, recall.
- **En el reporte de clasificación, ¿qué representan precisión, recall y f1-score?**
Precision: $TP / (TP + FP)$
Recall: $TP / (TP + FN)$
F1-score: Media armónica entre precision y recall
Evalúan balance entre detección correcta y errores, clave en desequilibrios de clases.
TP(true positives): verdaderos positivos: Son los casos que el modelo predijo como positivos y que en realidad sí eran positivos : Son los casos que el modelo predijo como positivos, pero en realidad eran negativos.
FP(false positives): falsos positivos : Son los casos que el modelo predijo como negativos, pero en realidad eran positivos.
FN(false negatives): falsos negativos : Son los casos que el modelo predijo como negativos, pero en realidad eran positivos.
TN: True Negatives (Verdaderos Negativos) : Son los casos que el modelo predijo como negativos y que en realidad sí eran negativos.

5. TensorBoard y Logging

- **¿Qué ventajas tiene usar TensorBoard durante el entrenamiento?**
Visualización en tiempo real de loss, accuracy, histogramas de pesos, imágenes, etc. Ideal para depuración y monitoreo. Además, permite comparar múltiples experimentos fácilmente desde la UI y exportar métricas.
- **¿Qué diferencias hay entre logear add_scalar, add_image y add_text?**
add_scalar: curvas de métricas (loss, acc...)
add_image: muestra imágenes (e.g. con predicciones)
add_text: logea etiquetas, descripciones o resultados
- **¿Por qué es útil guardar visualmente las imágenes de validación en TensorBoard?**
Para ver predicciones cualitativas, detectar fallos visibles, entender errores.
- **¿Cómo se puede comparar el desempeño de distintos experimentos en TensorBoard?**
Usando run names diferentes y luego comparar runs en TensorBoard (seleccionando múltiples experimentos en la UI).

6. Generalización y Transferencia

- **¿Qué cambios habría que hacer si quisiéramos aplicar este mismo modelo a un dataset con 100 clases?**
Cambiar num_classes en la última capa del modelo: Linear(..., 100)
- **¿Por qué una CNN suele ser más adecuada que una MLP para clasificación de imágenes?**
Porque aprende patrones espaciales locales (bordes, texturas), tiene menos parámetros gracias al peso compartido y respeta la estructura 2D.
- **¿Qué problema podríamos tener si entrenamos este modelo con muy pocas imágenes por clase?**
Riesgo de overfitting. El modelo puede memorizar. Se soluciona con:
Data augmentation
Transfer learning
Regularización
- **¿Cómo podríamos adaptar este pipeline para imágenes en escala de grises?**
Cambiar in_channels=1 en la red, y transformar las imágenes a 1 canal (e.g., usando albumentations.ToGray() o modificando el dataset).

7. Regularización

Preguntas teóricas:

- **¿Qué es la regularización en el contexto del entrenamiento de redes neuronales?**

Es una técnica que ayuda a prevenir el overfitting, es decir, cuando el modelo aprende demasiado bien los datos de entrenamiento y falla al generalizar a datos nuevos. Regularizar implica imponer restricciones al modelo (como limitar la magnitud de los pesos o introducir ruido) para fomentar una mejor generalización.

- **¿Cuál es la diferencia entre Dropout y regularización L2 (weight decay)?**

Dropout: Desactiva aleatoriamente un porcentaje de neuronas durante el entrenamiento, forzando a la red a no depender excesivamente de ninguna neurona en particular.

L2 / Weight Decay: Penaliza pesos grandes agregando una suma de los cuadrados de los pesos al loss, lo que empuja al modelo a tener pesos más pequeños.

- **¿Qué es BatchNorm y cómo ayuda a estabilizar el entrenamiento?**

Batch Normalization (BN) normaliza las activaciones de cada mini-batch a media cero y varianza uno, estabilizando la distribución de entrada de cada capa. Esto reduce el problema del internal covariate shift, acelerando el aprendizaje y permitiendo usar tasas de aprendizaje más altas.

- **¿Cómo se relaciona BatchNorm con la velocidad de convergencia?**

BN acelera la convergencia del entrenamiento al mantener constantes las distribuciones de activación a través de las capas. Esto permite que el modelo alcance una buena solución más rápidamente.

- **¿Puede BatchNorm actuar como regularizador? ¿Por qué?**

Sí. BN introduce cierto ruido debido a la variabilidad entre batches, lo que tiene un efecto regularizador similar al Dropout. Reduce la necesidad de usar Dropout en algunos casos, aunque se pueden combinar.

- **¿Qué efectos visuales podrías observar en TensorBoard si hay overfitting?**

Training loss sigue bajando, pero validation loss empieza a subir.

Accuracy de entrenamiento alta y accuracy de validación estancada o bajando.

Gráficas con una divergencia clara entre métricas de entrenamiento y validación.

También puede verse que la varianza de loss entre epochs en validación aumenta, o que ciertas clases presentan una caída en precisión.

- **¿Cómo ayuda la regularización a mejorar la generalización del modelo?**

La regularización limita la complejidad del modelo o introduce ruido estructurado, lo que fuerza al modelo a aprender patrones más robustos y

menos específicos del set de entrenamiento. Esto mejora su rendimiento en datos nuevos.

Actividades de modificación:

- **Agregar Dropout en la arquitectura MLP:**
Insertar capas `nn.Dropout(p=0.5)` entre las capas lineales y activaciones.
Comparar los resultados con y sin Dropout.
- **Agregar Batch Normalization:**
Insertar `nn.BatchNorm1d(...)` después de cada capa Linear y antes de la activación:

```
self.net = nn.Sequential(  
    nn.Flatten(),  
    nn.Linear(in_features, 512),  
    nn.BatchNorm1d(512),  
    nn.ReLU(),  
    nn.Dropout(0.5),  
    nn.Linear(512, 256),  
    nn.BatchNorm1d(256),  
    nn.ReLU(),  
    nn.Dropout(0.5),  
    nn.Linear(256, num_classes)  
)
```
- **Aplicar Weight Decay (L2):**
Modificar el optimizador:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001,  
weight_decay=1e-4)
```
- **Reducir overfitting con data augmentation:**
Agregar transformaciones en `Albumentations` como `HorizontalFlip`, `BrightnessContrast`, `ShiftScaleRotate`.
- **Early Stopping (opcional):**
Implementar un criterio para detener el entrenamiento si la validación no mejora después de N épocas.

Preguntas prácticas:

- **¿Qué efecto tuvo BatchNorm en la estabilidad y velocidad del entrenamiento?**
Estabiliza las activaciones internas reduciendo el "internal covariate shift", lo que hace que el entrenamiento sea más estable.
Acelera la convergencia, permitiendo usar tasas de aprendizaje más altas sin que se vuelva inestable.
Se observó un descenso más suave y consistente de la loss, con menor variabilidad entre épocas.

Reduce el zigzag en las curvas de pérdida y mejora la propagación del gradiente.

- **¿Cambió la performance de validación al combinar BatchNorm con Dropout?**

Sí, hubo una mejora en la generalización cuando se combinaron BatchNorm y Dropout de forma adecuada.

La mejor estructura observada fue:

Linear BatchNorm ReLU Dropout

Sin embargo, usar ambos en exceso puede sobre-regularizar, afectando negativamente la performance.

En general, la validación mejoró, especialmente en conjuntos con alta variabilidad o pocos datos.

- **¿Qué combinación de regularizadores dio mejores resultados en tus pruebas?**

La combinación que dio mejores resultados fue:

Batch Normalization + Dropout + Weight Decay

Observaciones:

BatchNorm por sí solo ya aportó mejoras significativas en estabilidad y precisión.

Dropout ayudó a prevenir el sobreajuste cuando se usó junto con BatchNorm.

Weight Decay (L2 regularization) complementó bien a los anteriores, penalizando pesos grandes y ayudando en la generalización.

- **¿Notaste cambios en la loss de entrenamiento al usar BatchNorm?**

Sí:

La curva de loss fue más suave, con menos oscilaciones entre batches.

Hubo un descenso sostenido de la pérdida sin caídas abruptas ni explosiones.

Se observaron menos colapsos del gradiente (valores de loss NaN o infinitos).

En general, el modelo se entrenó de forma más predecible y con menor necesidad de ajustar la tasa de aprendizaje manualmente.

8. Inicialización de Parámetros

Preguntas teóricas:

- **¿Por qué es importante la inicialización de los pesos en una red neuronal?**

La inicialización afecta si el modelo logra converger o se estanca.

Influye en la velocidad de entrenamiento y en la propagación del gradiente a través de las capas.

Una mala inicialización puede causar:

Desvanecimiento o explosión del gradiente.

Aprendizaje muy lento.

Salidas saturadas en activaciones como sigmoid o tanh.

- **¿Qué podría ocurrir si todos los pesos se inicializan con el mismo valor?**

Las neuronas pierden su diversidad, aprenden exactamente lo mismo.

Esto rompe la asimetría, lo que significa que:

Todas las neuronas hacen el mismo cálculo.

El modelo no aprende representaciones ricas o útiles.

Resultado: la red no entrena correctamente y la precisión se estanca.

- **¿Cuál es la diferencia entre las inicializaciones de Xavier (Glorot) y He?**

La inicialización Xavier (también llamada Glorot) y la inicialización He se diferencian principalmente en el tipo de funciones de activación para las que están diseñadas y en cómo calculan la varianza de los pesos. Xavier está pensada para funciones activación como tanh y sigmoid, y busca mantener la varianza constante tanto en la propagación hacia adelante como en la retropropagación, utilizando una varianza igual a 2 dividida por la suma de las entradas y salidas de la capa. En cambio, la inicialización de He está diseñada para activaciones como ReLU o LeakyReLU, que anulan los valores negativos y por lo tanto tienden a reducir la varianza. Para compensar esto, He utiliza una varianza igual a 2 dividida sólo por el número de entradas de la capa (f_{an_in}), lo que mantiene una propagación más estable del gradiente.

- **¿Por qué en una red con ReLU suele usarse la inicialización de He?**

Porque ReLU pone a cero todas las salidas negativas, lo que puede reducir la varianza de activación.

La inicialización de He mantiene la varianza estable al compensar esta reducción.

Esto mejora la propagación del gradiente y acelera el entrenamiento en redes profundas con ReLU.

- **¿Qué capas de una red requieren inicialización explícita y cuáles no?**

Las capas que requieren inicialización explícita en una red neuronal son principalmente las que tienen pesos entrenables, como las capas Linear y Conv2d, ya que sus pesos y sesgos (bias) deben iniciarse con valores adecuados para permitir un entrenamiento eficiente. En cambio, las capas como BatchNorm y Dropout no requieren inicialización manual. En el caso de BatchNorm, PyTorch ya inicializa los parámetros de escala (γ) y desplazamiento (β) por defecto con valores que no afectan inicialmente la salida ($\gamma = 1$, $\beta = 0$). Y Dropout no tiene parámetros entrenables, por lo que tampoco necesita inicialización.

Actividades de modificación:

- **Agregar inicialización manual en el modelo:**

En la clase MLP, agregar un método `init_weights` que inicialice cada capa:


```
def init_weights(self):
    for m in self.modules():
        if isinstance(m, nn.Linear):
            nn.init.kaiming_normal_(m.weight)
            nn.init.zeros_(m.bias)
```

- **Probar distintas estrategias de inicialización:**

Xavier (nn.init.xavier_uniform_)

He (nn.init.kaiming_normal_)

Aleatoria uniforme (nn.init.uniform_)

Comparar la estabilidad y velocidad del entrenamiento.

- **Visualizar pesos en TensorBoard:**

Agregar esta línea en la primera época para observar los histogramas:

```
for name, param in model.named_parameters():
    writer.add_histogram(name, param, epoch)
```

Preguntas prácticas:

- **¿Qué diferencias notaste en la convergencia del modelo según la inicialización?**

La inicialización tiene un impacto directo en la velocidad y estabilidad de la convergencia:

Kaiming (He): diseñada para activaciones ReLU, suele mostrar una convergencia rápida y estable, especialmente en redes profundas. Proporciona buenos gradientes al inicio del entrenamiento.

Xavier (Glorot): mejor con activaciones tanh o sigmoid, pero también funciona bien con ReLU si no hay muchas capas. Puede tardar un poco más en converger comparado con Kaiming, pero de forma estable.

Uniforme aleatoria (nn.init.uniform_): puede llevar a convergencia más lenta o ruidosa, especialmente si los rangos no están bien ajustados. A veces empieza con gradientes pequeños o muy desbalanceados.

En resumen:

- **Kaiming** → mejor rendimiento inicial con ReLU.
- **Xavier** → buena generalidad, algo más lenta.
- **Uniforme** → más sensible, requiere tuning.

- **¿Alguna inicialización provocó inestabilidad (pérdida muy alta o NaNs)?**

Sí, en general:

Las inicializaciones no adaptadas a la arquitectura o la activación (como usar Xavier o Uniform con muchas capas ReLU profundas sin batch norm) pueden provocar:

Explosión o desaparición de gradientes.

Pérdidas muy altas al inicio.

NaNs si los pesos iniciales son muy grandes y las activaciones se saturan.

El uso de BatchNorm y Dropout ayuda a mitigar estas inestabilidades, pero una mala inicialización aún puede afectar la estabilidad.

- **¿Qué impacto tiene la inicialización sobre las métricas de validación?**

La inicialización no cambia la capacidad del modelo final, pero influye en el camino que sigue durante el entrenamiento.

Una buena inicialización permite:

Llegar más rápido a una buena solución.

Evitar caer en mínimos pobres o en zonas planas.

Mejor estabilidad entre epochs (menos variación en val_loss y val_acc).

Una mala inicialización puede llevar a:

Sobreajuste temprano si hay gran desbalance en pesos.

Pérdidas altas de validación si no se logra una buena representación.

En modelos MLP, esto es crítico porque no tienen estructura inductiva como CNNs, por lo que dependen mucho del punto de partida.

- **¿Por qué bias se suele inicializar en cero?**

Porque el bias no depende del tamaño de entrada ni de la activación, sólo desplaza el resultado de la suma ponderada.

Inicializar en cero:

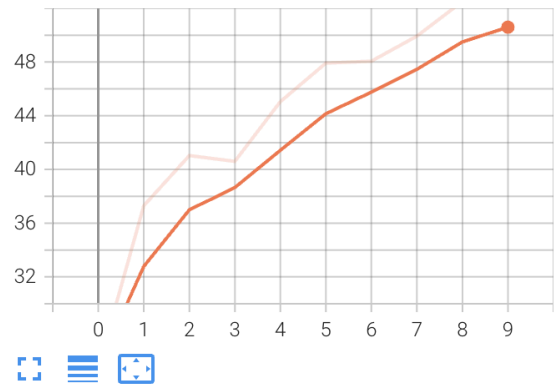
Evita introducir sesgos innecesarios al comienzo del entrenamiento.

No rompe la simetría, a diferencia de los pesos (los pesos deben tener diversidad desde el inicio, los bias no).

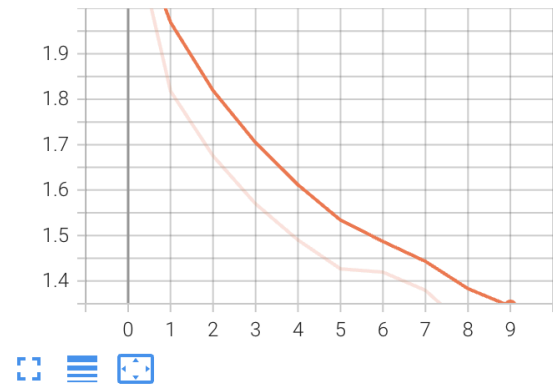
Si se usa BatchNorm, muchas veces el bias se vuelve redundante, ya que el batchnorm tiene su propio parámetro de desplazamiento.

Gráficos Finales:

train/accuracy
tag: train/accuracy

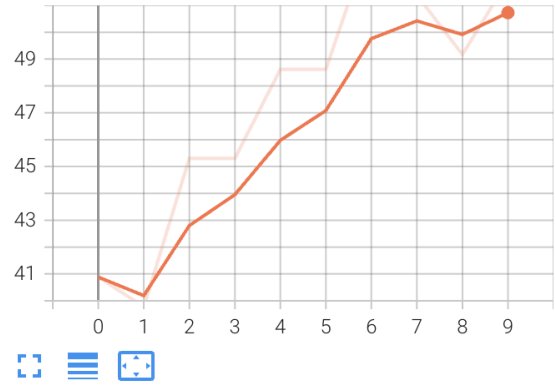


train/loss
tag: train/loss

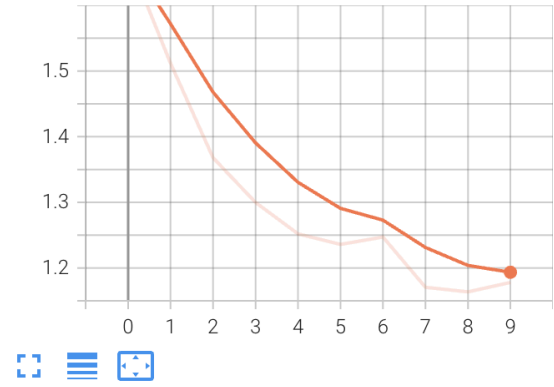


val

val/accuracy
tag: val/accuracy



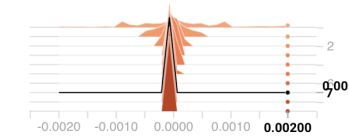
val/loss
tag: val/loss



Histogramas:

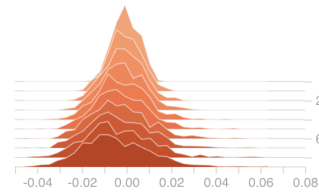
net.1.bias

net.1.bias
tag: net.1.bias



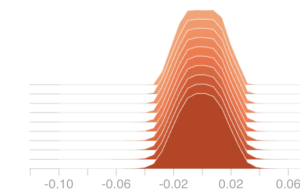
net.2.bias

net.2.bias
tag: net.2.bias



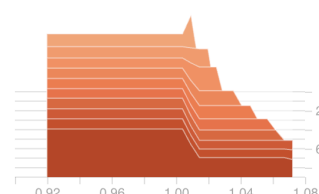
net.1.weight

net.1.weight
tag: net.1.weight



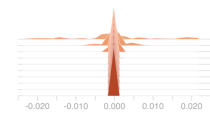
net.2.weight

net.2.weight
tag: net.2.weight



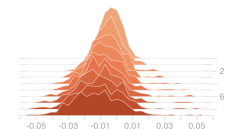
net.5.bias

net.5.bias
tag: net.5.bias



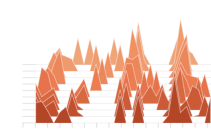
net.6.bias

net.6.bias
tag: net.6.bias



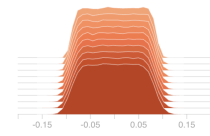
net.9.bias

net.9.bias
tag: net.9.bias



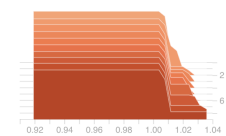
net.5.weight

net.5.weight
tag: net.5.weight



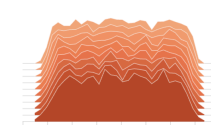
net.6.weight

net.6.weight
tag: net.6.weight



net.9.weight

net.9.weight
tag: net.9.weight



Distribuciones:

